A Experiment details

A.1 Datasets, splits, preprocessing, data augmentation

CIFAR-10 The CIFAR-10 dataset Krizhevsky [2012] consists of 60000 32x32 colour images in 10 classes, with 6000 images per class. There are 50000 training images and 10000 test images. We used the canonical train–test split. As a preprocessing, we normalized the images with the means (0.4914, 0.4822, 0.4465) and standard deviations (0.2023, 0.1994, 0.2010) for the three RGB channels, respectively. As augmentation, we used random horizontal flip, and 32x32 sized random crop from the zero padded 40x40 inputs.

CelebA The CelebA dataset Liu et al. [2015] is a large-scale face attributes dataset, it consists of 202599 number of images depicting faces of celebrities, each with 40 attribute annotations. We used a random split with 80% train and 20% test set sizes. Originally the CelebA images are sized 178×218 (width $\times$ height). As a preprocessing, we first reshaped these images to 256x256 pixels and then applied a center crop to 224x224 which was our final input shape for the Inception V1 networks. We subtracted 117 from each pixel. As augmentation, we applied a random rotation between degrees of -10 and 10.

A.2 Network architectures and training details

Tiny-10 Tiny-10 is a simple multi-layer convnet architecture. We use this model to provide a simple convnet for the experiments that trains fast on modern hardware. (A similar architecture was used also in Kornblith et al. [2019].) Table 1 details the layers along with the names we used in the paper to refer to a particular part of the network. While this naming is ambiguous as it could refer to three different activation spaces (in a row of the table), we use the activations after the batch normalization and before the nonlinearity if not otherwise stated.

We trained the model on CIFAR-10 for 300 epochs, the optimizer was SGD with Nesterov momentum 0.9. There was a schedule for the learning rate: started with the value of 0.1 and it was divided by 10 at 1/3 of the training, and with another 10 at the 2/3 of the training. The batch size was 128. We used weight decay with value 10^{-4}. The average accuracy of the resulting models was 86.55%.

Table 1: The Tiny10 architecture.

Tiny10	
Layers	Name
3×3 conv. 16-BN-ReLu	Layer 1
3×3 conv. 16-BN-ReLu	Layer 2
3×3 conv. 32 stride 2-BN-ReLu	Layer 3
3×3 conv. 32-BN-ReLu	Layer 4
3×3 conv. 32-BN-ReLu	Layer 5
3×3 conv. 64 stride 2-BN-ReLu	Layer 6
3×3 conv. 64 BN-ReLu	Layer 7
1×1 conv. 64-BN-ReLu	Layer 8
Global average pooling	
Dense	
Logits	

ResNet-20 Our ResNet-20 He et al. [2016] variant follows common practices regarding CIFAR-10: we use a 3-level architecture with three residual blocks per level. A residual block contains the following layers: Conv-Batchnorm-ReLU-Conv-Batchnorm. After each residual block, a ReLU operation follows the addition operation. Convolution kernels are sized 3x3. In the paper, we use the following naming convention: `LayerX.Y` corresponds to activations after the addition operation following a residual block with index `Y` in level `X`, with the exception of `Layer0.0`, which corresponds to the activation space after the first Conv-Batchnorm layer in the network preceding the residual blocks.

We trained the model on CIFAR-10 for 300 epochs, the optimizer was SGD with Nesterov momentum 0.9. There was a schedule for the learning rate: started with the value of 0.1 and it was divided by 10 at 1/3 of the training, and with another 10 at the 2/3 of the training. The batch size was 128. We used weight decay with value 10^{-4}. The average accuracies were 91.95%, 93.97%, 94.59%, 94.77% for the 1-width, 2-width, 3-width, and 4-width ResNets, respectively.

Inception V1 For the Inception V1 Szegedy et al. [2015], we omit the detailed description of the architecture as a reiteration would be cumbersome, and there are no specifics for the task at hand. Regarding the layer names, we follow the standard naming conventions.

We trained the model on CelebA for 20 epochs, using the Adam optimizer with parameters $\beta_1 = 0.9$ and $\beta_2 = 0.999$. Learning rate was 0.0001, batch size was 128.

A.3 Experiment 1 - Details for matching with least squares and task loss matching

In this experiment, we take network pairs (Model 1 and Model 2) which are of the same architecture but trained from different weight initializations and with different orderings of the training set.

Least squares matching Let A and B denote the activation matrices for the training data of Model 1 and Model 2, respectively. We appended an all ones vector to the activation matrix A of Model 1 to represent the bias. Then we calculated the pseudoinverse $A^\dagger$ of A using SVD. The transformation matrix and the bias of the stitching layer was obtained by calculating $A^\dagger B$.

Task loss matching We initialized the transformation matrix and the bias of the stitching layer to the least squares solution (which was calculated as described above). Then we trained the stitching layer on the train set for 30 epochs. The utilized loss was cross-entropy to Model 2 activations. The optimizer was Adam with parameters $\beta_1 = 0.9$ and $\beta_2 = 0.999$, learning rate was set to 10^{-3}, batch size was 128.

We used the following hyperparameter selection protocol: after a grid search with Tiny-10 and ResNet consisting of the parameter settings { Optimizer: Adam, SGD } $\times$ { Learning rate: 0.1, 0.01, 0.001, 0.0001, 0.00001 } and training for 300 epochs, we observed that Adam is significantly better than SGD, and that learning rates below 0.001 do not affect performance, only prolong the training time. Moreover, we observed that with the selected hyperparameters the training of the stitching layer always reaches a plateau before the 30th epoch, thus, we set this hyperparameter accordingly.

On Tiny-10, we matched the activations after the batch normalization layer (which comes after the convolution and before the nonlinearity). With ResNets, we plotted the results that correspond to stitchings after the addition operations of the residual blocks. (Matchings on the inside layers of residual blocks are harder to interpret, while the results are very similar).

A.4 Experiment 2 - Details for networks of different width

We utilized the same methodology and settings to train the stitching layer as described in Appendix A.3. The baseline 3-level ResNet had 16, 32, and 64 filters in the convolution layers for each level, respectively. The networks of different width were obtained by multiplying these baseline filter numbers with the width multiplier.

A.5 Experiment 3 - Details for similarity indices and task performance

We utilized the same methodology and settings to set or train the stitching layer as described in Appendix A.3.

CKA, CCA and SVCCA is calculated on the whole validation set. We can fit this amount of data into memory without needing to resort to sampling methods like "minibatch CKA" Nguyen et al. [2021]. We note that the resulting CKA values are indistinguishable from values obtained when working with any number of data points between 2500 and 10000.

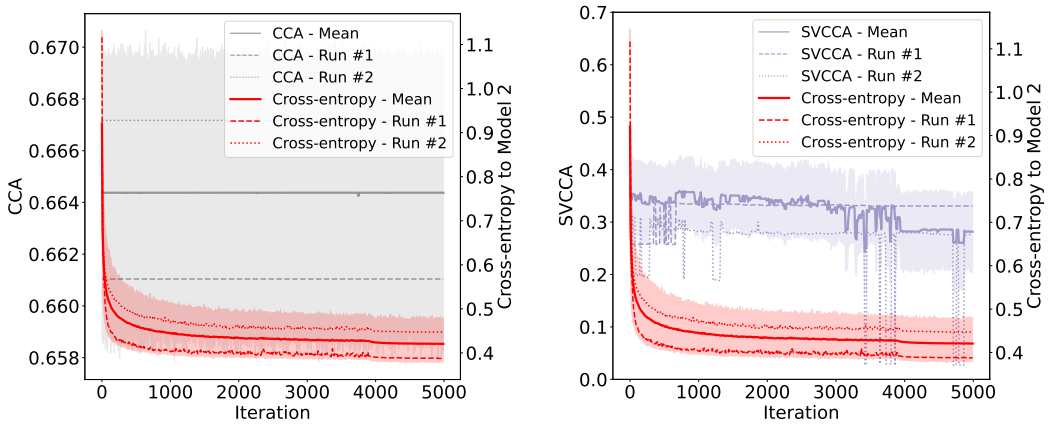

Figure 9: CCA, SVCCA and cross-entropy values over the training iterations of task loss matching started from the optimal Least Squares matching. Tiny-10 network, stitching at Layer 3. Bold lines are averages of 10 runs, with bands representing standard deviations; dotted and dashed lines correspond to two specific runs to show the individual characteristics of the training of the stitching layer.

A.6 Experiment 4 - Details for low CKA value with high accuracy

The loss penalizing CKA was calculated on the training minibatch, the reported values were calculated on the whole validation set. We trained the stitching layer for 2 epochs with batch size 128, CKA loss weight 0.1, the optimizer was Adam with parameters $\beta_1 = 0.9$ and $\beta_2 = 0.999$, learning rate was 0.001 at the first epoch and 0.0001 in the second epoch. (Nevertheless, we observed that the general outcome of the experiment is quite robust for a large range of hyperparameters.)

A.7 Experiment 5: Details for dependence of accuracy on initializations

We train the stitching layer on task loss from 50 random and 50 optimal least squares initializations between two Tiny-10 architectures trained from different initializations. Other settings of the experiment are the same as presented in Appendix A.3. Figure 10 shows the results for all layers, between three pairs of networks.

A.8 Experiment 6: Details for linear mode connectivity of transformation matrices

We utilized the same methodology and settings to train the stitching layers as described in Appendix A.3. See Figure 11 for detailed results.

A.9 Experiment 7: Details for sparsity — direct vs. task loss matching

To compare the sparsity tolerance of task loss matching and direct matching, we trained 5 stitching layers with both methods until convergence, which means 30 and 200 epochs, respectively. We used the Tiny-10 architecture, and the CIFAR-10 dataset. For task loss matching we utilized the same methodology and settings as described in Appendix A.3. We trained the direct matching using the Adam optimizer with parameters $\beta_1 = 0.9$ and $\beta_2 = 0.999$, learning rate was set to 10^{-2}. We ran the trainings with different L1-regularization coefficients, namely: 0, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 10^{0}, 10^{1}, 10^{2}, 10^{3}, 10^{4} to achieve increasing sparsity. In order to achieve an actual sparse matrix, we set every entry of the matrix to 0 below a certain threshold, 10^{-4}. We evaluated the performance of the sparse stitching matrices on the task. Figure 12 shows the relative accuracy and sparsity (ratio of zero elements) with respect to L1-regularization term, and Figure 13 shows the relative accuracy with respect to sparsity for all matched layers.